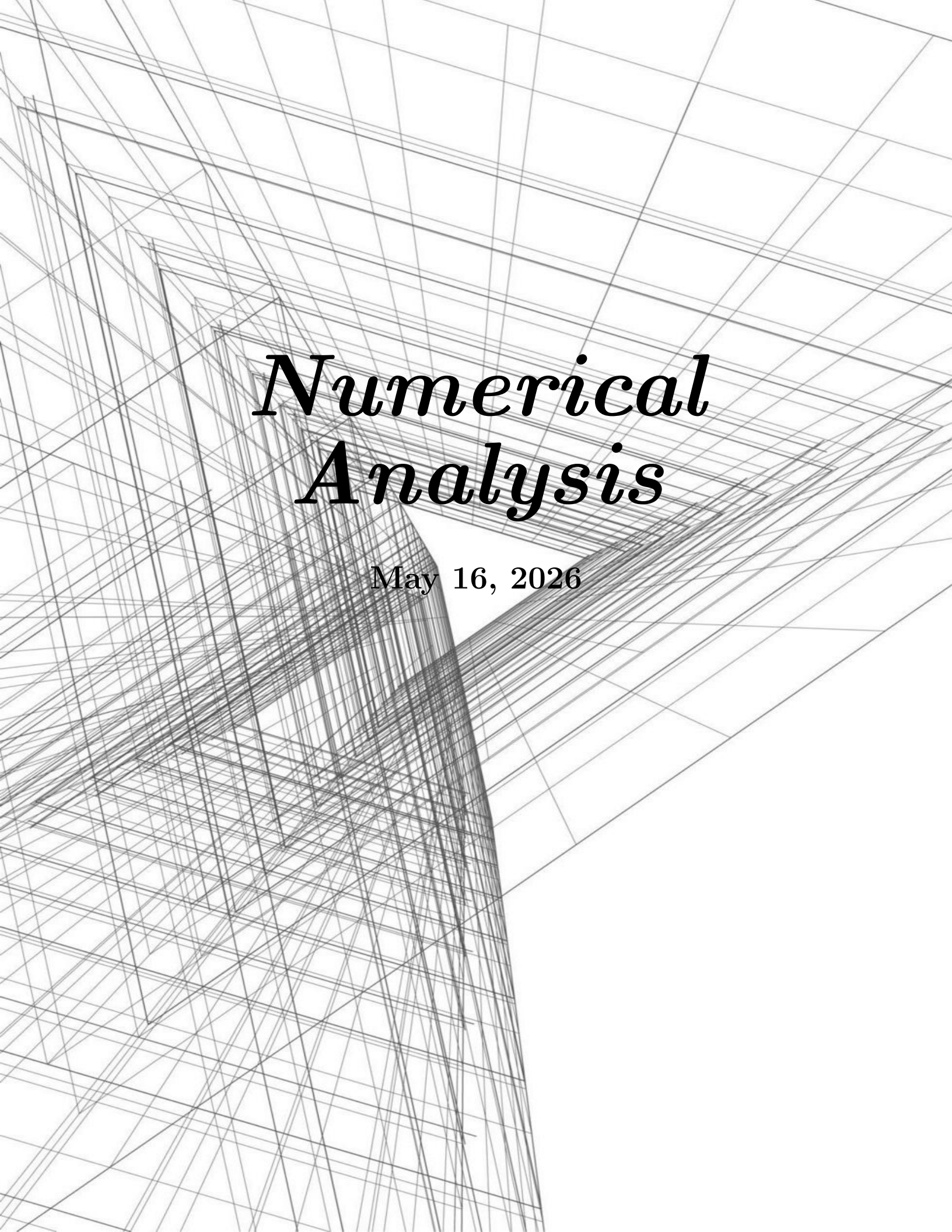

Numerical Analysis

May 16, 2026

Contents

1 Preliminaries **1**

1.1 Error Analysis . . . 1

1.1.1 Binary Machine Numbers . . . 1

1.1.2 Decimal Machine Numbers . . . 2

1.1.3 Finite Digit Arithmetic . . . 3

1.1.4 Nested Arithmetic . . . 4

1.2 Algorithms and Convergence . . . 4

1.2.1 Characterization of Algorithms . . . 4

1.2.2 Rate of Convergence . . . 5

2 Numerical Solution of Linear Systems **7**

2.1 Gaussian Elimination . . . 7

2.1.1 Operational Counts . . . 9

2.1.2 A Priori Error Estimates and Condition Number . . . 10

Chapter 1

Preliminaries

1.1 Error Analysis

The arithmetic performed by a calculator or computer is not exact. In the computational world, each representable number has only a fixed and finite number of digits. This means, for example, that only some of the rational numbers can be represented exactly.

The error that is produced when a calculator or computer is used to perform real number calculations is called **round-off error**. In a computer, only a relatively small subset of the real number system is used for the representation of all the real numbers. This subset contains only rational numbers, both positive and negative, and stores the fractional part, together with an exponential part.

1.1.1 Binary Machine Numbers

IEEE 754-2008 provides standards for binary and decimal floating point numbers, formats for data interchange, algorithms for rounding arithmetic operations, and the handling of exceptions. Formats are specified for single, double, and extended precisions, and these standards are widely used in computer hardware and software.

A 64-bit (binary digit) representation is used for a real number. The first bit s is the sign bit, the next 11 bits are used for the exponent c, called the **characteristic**, and the remaining 52 bits are used for the fraction f, called the **mantissa**. To save storage and provide a unique representation for each floating-point number, a normalization is imposed.

A number with $c \neq 0$ is called a **normalized number**, and is represented as

$$N = (-1)^s 2^{c-1023}(1+f), \tag{1.1}$$

This represents a number in the range

$$[N - 2^{-52}, N + 2^{-52})$$

Numbers occurring in calculations that have a magnitude less than $2^{-1022}(1+0)$ results in a **underflow**, and is generally set to zero. Numbers with a magnitude greater than $2^{1023}(2-2^{-52})$ results in an **overflow**, and generally causes an error message to be generated.

When $c = 0$ and $f \neq 0$, the number is called a **denormalized number**, and is represented as

$$N = (-1)^s 2^{-1022} f. \tag{1.2}$$

zero is represented by $s = 0, 1, c = 0, f = 0$, and is called a **signed zero**.

1.1.2 Decimal Machine Numbers

The use of binary digits imposes difficulties to examine the problems from finite digit representation. So we shall use a normalized decimal representation of a real number, with the form

$$N = \pm 0.d_1 d_2 \cdots d_k \times 10^n, \qquad 1 \leqslant d_1 \leqslant 9, \quad 0 \leqslant d_i \leqslant 9, i = 2, 3, \cdots, k,$$

This number is called k-digit decimal machine number.

From analysis on the real line, there is a way to normalize any positive real number to the form

$$y = 0.d_1 d_2 \cdots d_k \cdots \times 10^n,$$

The floating point representation of y is obtained by terminating the mantissa after k digits, and is denoted by $fl(y)$. There are two common methods for terminating the mantissa:

- **Chopping**: simply drop off the digits after the k-th digit.
- **Rounding**: if $d_{k+1} \geqslant 5$, then add 10^{-k} to the chopped number; otherwise, simply chop off the digits after the k-th digit.

The error caused by replacing y by $fl(y)$ is called the **round-off error**. There are mainly two origins of errors:

- **Original error**: the error caused by the initial approximation of a number.
- **Round-off error**: the error caused by the finite digit representation of a number. (Rounding error and Chopping error are two types of round-off errors.)

Definition 1.1.1: Error

Suppose p^* is an approximation to a number p. The **actual error** is defined as $p - p^*$. The **absolute error** is defined as $|p - p^*|$. The **relative error** is defined as $\dfrac{|p - p^*|}{|p|}$, provided $p \neq 0$.
An error bound is a nonnegative number larger than the absolute error.

Remark:
We often cannot find an accurate value for the true error in an approximation. Instead, we find a bound for the error, which gives us a "worst-case" error.

Definition 1.1.2: Significant Digits

The number p^* is said to approximate p to t **significant digits**, if t is the largest nonnegative integer such that

$$\frac{|p - p^*|}{|p|} \leqslant 5 \times 10^{-t}. \tag{1.3}$$

The floating point representation of a number y has relative error $\dfrac{|y - fl(y)|}{|y|}$. If k-decimal chopping is used, then

$$\frac{|y - fl(y)|}{|y|} = \left| \frac{0.d_{k+1} d_{k+2} \cdots \times 10^{n-k}}{0.d_1 d_2 \cdots d_k \times 10^n} \right| \leqslant \frac{1}{0.1} \times 10^{-k} = 10^{1-k}.$$

So 10^{1-k} is a error bound for the relative error of $fl(y)$.

Similarly, $0.5 \times 10^{1-k}$ is a error bound for the relative error of $fl(y)$ when k-decimal rounding is used.

1.1.3 Finite Digit Arithmetic

Assume that $x, y \in \mathbb{R}$, define symbols $\oplus, \ominus, \otimes, \oslash$ for the finite digit arithmetic as follows:

$$x \oplus y = fl(fl(x) + fl(y)), \qquad x \ominus y = fl(fl(x) - fl(y)),$$
$$x \otimes y = fl(fl(x) \cdot fl(y)), \qquad x \oslash y = fl\left(\frac{fl(x)}{fl(y)}\right).$$

Some calculation would produce a large error. For example, the cancellation of significant digits due to the subtraction of nearly equal numbers. Suppose $x > y$ have the k-digit representations:

$$fl(x) = 0.d_1 d_2 \cdots d_p \alpha_{p+1} \cdots \alpha_k \times 10^n,$$
$$fl(y) = 0.d_1 d_2 \cdots d_p \beta_{p+1} \cdots \beta_k \times 10^n,$$

where $\alpha_{p+1} > \beta_{p+1}$. Then we have

$$\begin{aligned} x \ominus y &= fl(fl(x) - fl(y)) \\ &= fl\left(0.00 \cdots 0 \sigma_{p+1} \cdots \sigma_k \times 10^n\right) \\ &= 0.\sigma_{p+1} \cdots \sigma_k \times 10^{n-p}, \end{aligned}$$

which has at most $k - p$ significant digits. In any calculation device, the last few digits of $x - y$ would be assigned either zero or some random digits, which may lead to a large relative error in afterwards calculations (for example, dividing by a small number).

Sometimes, the loss of accuracy can be avoided by reformulating the calculation. For example, to compute the root of a quadratic equation $ax^2 + bx + c = 0$ with the root formula:

Example: **Reformulation of Calculation**

Consider $x^2 + 62.1x + 1 = 0$. The roots are given by

$$x_1 \approx -0.01610723, \qquad x_2 \approx -62.08389277.$$

Note that directly calculating x_1 would cause a large error due to the cancellation of significant digits. Instead, we can calculate x_1 by a reformulation of the root formula:

$$x_1 = \frac{-b - \sqrt{b^2 - 4ac}}{2a} = \frac{-2c}{b + \sqrt{b^2 - 4ac}}$$

which would result in a much smaller relative error, from 2.4×10^{-1} to 6.2×10^{-4} for a 4-digit decimal machine number.

We can do some approximations on the relative errors as follows: Let x^*, y^* be the approximations of x, y respectively, with relative errors e_x, e_y. Then we have

- Addition and Subtraction: $\dfrac{|(x \pm y) - (x^* \pm y^*)|}{|x \pm y|} \leqslant \dfrac{|x|e_x + |y|e_y}{|x \pm y|}$.

1.1.4 Nested Arithmetic

This is a way to calculate a polynomial:

Consider the polynomial

$$p(x) = a_0 + a_1x + a_2x^2 + \cdots + a_nx^n. \tag{1.4}$$

We can calculate $p(x)$ by the following nested form:

$$p(x) = a_0 + x(a_1 + x(a_2 + \cdots + x(a_{n-1} + xa_n)\cdots)). \tag{1.5}$$

This reduces the number of multiplications from $n(n+1)/2$ to n, and thus significantly reduces the round-off error.

Remark:
Polynomials should always be expressed in nested form before performing an evaluation because this form minimizes the number of arithmetic calculations. This shows that one way to reduce round-off error is to reduce the number of computations.

1.2 Algorithms and Convergence

Definition 1.2.1: Algorithm

An **algorithm** is a finite sequence of instructions that, if followed, accomplishes a particular task. The task may be to solve a mathematical problem, or to perform a computation.

We use a pseudocode to describe the algorithms.

1.2.1 Characterization of Algorithms

A **stable** algorithm is one that produces correspondingly small changes in the output for small changes in the input. Some algorithm are **conditionally stable**, which means that the algorithm is stable for some inputs but not for others.

To quantify this idea, suppose an error with magnitude $E_0 > 0$ is introduced at some stage of the algorithm, and E_n is the magnitude of the error after n steps, then:

- If $E_n \approx CnE_0$ for some constant C, then the growth of the error is **linear**.
- If $E_n \approx C^nE_0$ for some constant $C > 1$, then the growth of the error is **exponential**.

Linear growth of error is usually unavoidable, and when C and E_0 are small, the results are generally acceptable. Exponential growth of error should be avoided because the term C^n becomes large for even relatively small values of n. This leads to unacceptable inaccuracies, regardless of the size of E_0. As a consequence, an algorithm that exhibits linear growth of error is stable, whereas an algorithm exhibiting exponential error growth is unstable.

Remark:
Usually when we analyze the stability of an algorithm, we only assume the loss of accuracy

occurs at the beginning of the algorithm (when input data is read into the computer), and neglect the round-off error produced by the arithmetic operations in the algorithm. This is a simplified model of the error, and the actual error is likely to be larger.

1.2.2 Rate of Convergence

Definition 1.2.2: Rate of Convergence

Suppose $\{\beta_n\}$ is a sequence converging to zero and $\{\alpha_n\}$ converges to α. If there exists a constant $K > 0$ such that

$$|\alpha_n - \alpha| \leqslant K|\beta_n|$$

for large n, then we say $\{\alpha_n\}$ converges to α with **rate of convergence** $O(\beta_n)$, and write $\alpha_n = \alpha + O(\beta_n)$.
Similarly for functions, if $\lim_{h\to 0} G(h) = 0$ and $\lim_{h\to 0} F(h) = L$, and there exists a constant $K > 0$ such that

$$|F(h) - L| \leqslant K|G(h)|$$

for small h, then we say $F(h)$ converges to L with **rate of convergence** $O(G(h))$, and write $F(h) = L + O(G(h))$.

Remark:
Generally we have the following conclusions for a good approximation algorithm:

- Avoid subtraction of nearly equal numbers. (Increase relative error.)
- Avoid division by small numbers. (Increase Absolute error.)
- Avoid adding a small number to a large number.
- Reduce the number of arithmetic calculations. (Increase round-off error.)
- Alogrithms should be stable.

Chapter 2

Numerical Solution of Linear Systems

Finding the solution of a linear algebraic equation system of large order and calculating the inverse of a matrix of large order can be difficult numerical tasks. The difficulties are practical and stem from

- the labor required in a lengthy sequence of calculations.
- the possible loss of accuracy in such lengthy calculations performed with a fixed number of decimal places.

Thus to determine the feasibility of solving a particular problem with given equipment, several questions should be answered:

- How many arithmetic operations are required to apply a proposed method?
- What will be the accuracy of a solution to be found by the proposed method (a priori estimate)?
- How can the accuracy of the computed answer be checked (a posteriori estimate)?

In the chapter we mainly discuss two problems:

- the solution of a linear system of equations $Ax = b$.
- the calculation of the inverse of a matrix A^{-1}.

These two problems are closely related: If A^{-1} is known, then the solution of $Ax = b$ can be found by a single matrix-vector multiplication $x = A^{-1}b$. Conversely, if the solution of $Ax = b$ can be found for any b, then the inverse of A can be found by solving n systems of equations $Ax = e_i$, where e_i is the i-th column of the identity matrix.

2.1 Gaussian Elimination

Consider the system of linear equations

$$Ax = f \tag{2.1}$$

where $A = (a_{ij})$ is an $n \times n$ matrix, $x = (x_1, x_2, \ldots, x_n)^T$ is the unknown vector, and $f = (f_1, f_2, \ldots, f_n)^T$ is the known vector.

Giving the system of equations, we perform the row operations to transform the system into an upper triangular form: Let the equation of k-th step be $A^{(k)}x = f^{(k)}$. We perform the following row operations:

$$a_{ij}^{(k)} = \begin{cases} a_{ij}^{(k-1)}, & i \leqslant k-1 \\ 0, & i \geqslant k, j \leqslant k-1 \\ a_{ij}^{(k-1)} - \frac{a_{i,k-1}^{(k-1)}}{a_{k-1,k-1}^{(k-1)}} a_{k-1,j}^{(k-1)}, & i \geqslant k, j \geqslant k \end{cases} \tag{2.2}$$

and for the right-hand side vector,

$$f_i^{(k)} = \begin{cases} f_i^{(k-1)}, & i \leqslant k-1 \\ f_i^{(k-1)} - \frac{a_{i,k-1}^{(k-1)}}{a_{k-1,k-1}^{(k-1)}} f_{k-1}^{(k-1)}, & i \geqslant k \end{cases} \tag{2.3}$$

These formulae represent the result of multiplying the $(k-1)$-th equation by $a_{i,k-1}^{(k-1)}/a_{k-1,k-1}^{(k-1)}$ and subtracting it from the i-th equation, for $i \geqslant k$. After $n-1$ steps, we have an upper triangular system of equations. It has been assumed above that the elements $a_{k-1,k-1}^{(k-1)}$ are nonzero for $k = 1, 2, \ldots, n-1$. This holds if the matrix A is nonsingular.

If this is not the case, we can perform row interchanges to make the pivot element nonzero.

Theorem 2.1.1: Gauss Elimination

Let A be be such that the Gaussian elimination procedure in natural order can be performed resulting in an upper triangular system of equations. Then A is nonsigular and in fact

$$\det A = a_{11}^{(1)} a_{22}^{(2)} \cdots a_{nn}^{(n)}. \tag{2.4}$$

The final matrix $A^{(n)} = U$ is an upper triangular matrix, and A has the LU factorization $A = LU$, where $L = m_{ik}$ is a unit lower triangular matrix with entries

$$m_{ik} = \begin{cases} 0, & i < k \\ 1, & i = k \\ a_{ik}^{(k)}/a_{kk}^{(k)}, & i > k \end{cases}$$

The final vector $f^{(n)} = g$ is $g = L^{-1}f$.

The system is equivalent to $Ux = g$, which can be solved by back substitution:

$$x_n = g_n/u_{nn}, \quad x_i = (g_i - \sum_{j=i+1}^{n} u_{ij}x_j)/u_{ii}, \quad i = n-1, n-2, \ldots, 1.$$

Theorem 2.1.2: Gauss Elimination with Pivoting

Let the matrix A has rank r. Then we can find a sequence of distinct row and column indices $(i_1, j_1), (i_2, j_2), \ldots, (i_r, j_r)$ such that the corresponding pivot elements in $A^{(1)}, A^{(2)}, \ldots, A^{(r)}$ are nonzero and that $a_{ij}^{(r)} = 0$ for $i \notin \{i_1, i_2, \ldots, i_r\}$. Let us define the permutation matrices, whose columns are unit vectors,

$$P = (e^{(i_1)}, e^{(i_2)}, \ldots, e^{(i_r)}, e^{(i_{r+1})}, \ldots, e^{(i_n)}), \quad Q = (e^{(j_1)}, e^{(j_2)}, \ldots, e^{(j_r)}, e^{(j_{r+1})}, \ldots, e^{(j_n)}).$$

where $i_1, i_2, \ldots, i_n$ and $j_1, j_2, \ldots, j_n$ are permutations of $1, 2, \ldots, n$. Then the system

$$By = g, \qquad B = P^T AQ, y = Q^T x, g = P^T f$$

is equivalent to the original system $Ax = f$ and can be reduced to an upper triangular system of equations by Gaussian elimination without pivoting.

In actual computations on digital computers it is a simple matter to record the order of the pivot indices (i_ν, j_ν) and to do the arithmetic accordingly.

Remark:
One way to pick the pivots is to require that (i_k, j_k) be the indices of a maximal coefficient in the system of $n - k + 1$ equations that remain at the k-th step. This method of selecting maximal pivots is recommended as being likely to introduce the least loss of accuracy in the arithmetical operations that are based on working with a finite number of digits.

Another commonly used pivotal selection method eliminates the variables $x_1, \ldots, x_{n-1}$ successively by selecting $(i_k, j_k) = (i_k, k)$, where i_k is the index of the largest element in the k-th column of the system of $n - k + 1$ equations that remain at the k-th step. This method of maximal column pivots is particularly convenient for use on an electronic computer if the large matrix of coefficients is stored by columns since the search for a maximal column element is then quicker than the maximal matrix element search.

2.1.1 Operational Counts

If the $n \times n$ matrix A is nonsingular, Gaussian elimination might be employed to solve the n special linear systems and thus to obtain A^{-1}. *However, we shall show here that for any value of m this procedure is less efficient than an appropriate application of Gaussian elimination to the m systems in question.*

The current convention is to count only multiplications and divisions. This custom arose because the first modern digital computers performed additions and subtractions much faster than they did multiplications and divisions which were done in comparable lengths of time.

Let us consider first the m systems, with arbitrary $f(j)$, where $j = 1, 2, \ldots, m$. We assume, for the operational count, that the elimination proceeds in the natural order. The most efficient application then performs the operations in equations (2.2) once for all m systems, and once for each of the m systems for the operations in equations (2.3). Thus, we find that it requires

$$\begin{aligned} (n-k+1)^2 + (n-k+1) \quad & \text{ops. for (2.2)} \\ (n-k+1) \quad & \text{ops. for (2.3)} \end{aligned}$$

to eliminate x_{k-1}. Thus, the total number of operations for the elimination phase is given by summing over k:

$$\begin{aligned} \frac{1}{3}n(n^2-1) \quad & \text{ops. for triangularization} \\ \frac{1}{2}n(n-1) \quad & \text{ops. for solving one inhomogeneous vector } f(j) \end{aligned}$$

To solve the resulting triangular system, we need $(n-i)$ multiplications and one division for x_i. So we have

$$\frac{1}{2}n(n+1) \qquad \text{ops. for back substitution}$$

In total we have

Lemma 2.1.1: Operational Count for Gaussian Elimination

The total number of multiplications and divisions required to solve m systems of linear equations by Gaussian elimination is

$$\frac{1}{3}n^3 + mn^2 - \frac{1}{3}n \tag{2.5}$$

To compute A^{-1}, just set $m = n$ and we get

$$\frac{4}{3}n^3 - \frac{1}{3}n \qquad \text{ops. for computing } A^{-1} \text{ by Gaussian elimination}$$

However, here the n inhomogeneous vectors are $e^{(j)}$, which is special. So we can reduce the number of operations with some tactics. That is, for any fixed $1 \leqslant j \leqslant n$, the calculations to be counted in (2.3) can start at $i = j$, which is $k = j + 2$. Thus, we have

$$\sum_{k=j+2}^{n} (n-k+1) = \frac{1}{2}(j^2 - (2n-1)j + n^2 - n)$$

to modify the inhomogeneous vector $e^{(j)}$ for $j = 1, 2, \ldots, n-2$, we have

$$\frac{1}{6}(n^3 - 3n^2 + 2n) \qquad \text{ops. for modifying the inhomogeneous vectors}$$

The other operations are the same as before, so we have

Lemma 2.1.2: Operational Count for Inversion

The total number of multiplications and divisions required to compute A^{-1} by Gaussian elimination is

$$n^3 \tag{2.6}$$

Thus, to solve m systems by inversion, we need

$$n^3 + mn^2 \qquad \text{ops. for solving } m \text{ systems by inversion}$$

2.1.2 A Priori Error Estimates and Condition Number

In the course of actually carrying out the arithmetic required to solve $Ax = f$ by any procedure, round-off errors will in general be introduced.

We recall that a computation is said to be well-posed if "small" changes in the data cause "small" changes in the solution. Now the data is the matrix A and the vector f, and the solution is the vector x. Suppose we have a perturbation:

$$(A + \delta A)(x + \delta x) = f + \delta f$$

Theorem 2.1.3: Condition Number

Let A be a nonsingular matrix. And

$$\|\delta A\| < \frac{1}{\|A^{-1}\|}$$

Then we have

$$\frac{\|\delta x\|}{\|x\|} \leqslant \frac{\mu}{1-\mu\delta A/\|A\|}\left(\frac{\|\delta A\|}{\|A\|}+\frac{\|\delta f\|}{\|f\|}\right) \tag{2.7}$$

where $\mu = \|A\|\|A^{-1}\|$ is called the condition number of A.

Proof. Since $\|A^{-1}\delta A\| \leqslant \|A^{-1}\|\|\delta A\| < 1$, we have $I + A^{-1}\delta A$ is nonsingular. Thus,

$$\|(I+A^{-1}\delta A)^{-1}\| \leqslant \frac{1}{1-\|A^{-1}\delta A\|} \leqslant \frac{1}{1-\mu\delta A/\|A\|}$$

from Taylor expansion. We also have

$$\delta x = (I+A^{-1}\delta A)^{-1}A^{-1}(\delta f - \delta A x)$$

Thus,

$$\frac{\|\delta x\|}{\|x\|} \leqslant \frac{\|A^{-1}\|}{1-\mu\delta A/\|A\|}\left(\frac{\|\delta f\|}{\|x\|}+\|\delta A\|\right)$$

Note that

$$\|f\| = \|Ax\| \leqslant \|A\|\|x\|$$

we have

$$\frac{\|\delta x\|}{\|x\|} \leqslant \frac{\mu}{1-\mu\delta A/\|A\|}\left(\frac{\|\delta A\|}{\|A\|}+\frac{\|\delta f\|}{\|f\|}\right)$$

giving the desired result. □

It shows that small relative changes in A and f would cause small relative changes in x if the factor

$$\frac{\mu}{1-\mu\delta A/\|A\|}$$

is not too large. Thus, it is clear that when the condition number μ is not too large, the system is well-conditioned. However, we shall note that there is an lower bound for μ:

$$\|I\| = \|AA^{-1}\| \leqslant \|A\|\|A^{-1}\| = \mu$$

We can apply the theorem to estimate the effects of round-off errors committed in solving linear systems by Gaussian elimination and other direct methods. But it is possible to view the computed solution as the exact solution, say $x+\delta x$, of a perturbed system, and the basic problem now is to determine the magnitude of the perturbations, δA and δf. This type of approach is called a *backward error analysis.* It is rather clear that there are many perturbations δA and δf that give the same result. Our analysis for the Gaussian elimination method will define δA and δf in a way that

$$\delta f = 0.$$

So that

$$\frac{\|\delta x\|}{\|x\|} \leqslant \frac{\mu\|\delta A\|/\|A\|}{1-\mu\|\delta A\|/\|A\|}$$

In the case of Gaussian elimination we have seen that exact calculations yield the factorization

$$LU = A$$

Now, lets consider the effect of round-off errors in the Gaussian elimination procedure from the perspective of LU factorization. The Gaussian elimination procedure can be viewed as computing $g = L^{-1}f$ and the back substitution as computing $x = U^{-1}g$.

The Gaussian elimination procedure In the phase, we calculate the entries of L and U by the formulae in (2.2). With exact calculations, we have $LU = A$. However, with finite precision arithmetic in these evaluations, we do not obtain exactly L and U, but say some triangular matrices $\mathcal{L}$ and $\mathcal{U}$. We define the perturbation E due to inexact calculations by

$$\mathcal{L}\mathcal{U} = A + E$$

The back substitution In the phase, we calculate the entries of x by the formulae in (2.3). With exact calculations, we have $Lg = f$ and $Ux = g$. However, with finite precision arithmetic, we have got $g + \delta g$ and $x + \delta x$ such that

$$\begin{aligned}(\mathcal{L} + \delta\mathcal{L})(g + \delta g) &= f,\\ (\mathcal{U} + \delta\mathcal{U})(x + \delta x) &= g + \delta g\end{aligned}$$

Here we use the $\mathcal{L}$ and $\mathcal{U}$ obtained in the Gaussian elimination procedure. We have

$$A + \delta A = (\mathcal{L} + \delta\mathcal{L})(\mathcal{U} + \delta\mathcal{U}), \qquad \delta A = E + \mathcal{L}(\delta\mathcal{U}) + (\delta\mathcal{L})\mathcal{U} + (\delta\mathcal{L})(\delta\mathcal{U})$$

Remark:
We can say that L and U took round-off errors both in the Gaussian elimination procedure and the back substitution, the first phase gives inexact $\mathcal{L}$ and $\mathcal{U}$, and the second phase gives additional perturbations $\delta\mathcal{L}$ and $\delta\mathcal{U}$ here.
This tactic is useful in analyzing algorithm errors: For inexact solution $Ax = f$, we say that the result we got $x + \delta x$ is the exact solution of a perturbed system $(A + \delta A)(x + \delta x) = f$, where we ASSUME (entirely mathematical model) that f is not perturbed and accurate, to eliminate a possible degree of freedom in the choice of δA and δf. Then we can analyze the error by estimating $\|\delta A\|$ and applying the condition number theorem.

Now, we must estimate $\|E\|, \|\delta\mathcal{L}\|$ and $\|\delta\mathcal{U}\|$ for the actual computations.

We shall assume that floating-point arithmetic operations are performed with a t-digit decimal mantissa and that the system has been ordered so that the natural order of pivots is used.

For a convenient notation, in place of the accurate matrices $A^{(k)}$ defined by (2.2), we shall use the matrices $B^{(k)} = (b_{ij}^{(k)})$ with the final such matrix $B^{(n)} = \mathcal{U} = (u_{ij})$. Similarly, the lower triangular matrix $\mathcal{L} = (s_{ij})$ is given by the computed multipliers. For simplicity, we assume that the given matrix elements a_{ij} are exactly representable in the floating-point system.

The floating-point calculations yield

- For $k = 1$,
$$b_{ij}^{(1)} = a_{ij} \qquad i, j = 1, 2, \ldots, n$$
- For $k = 1, 2, \ldots, n-1$,
$$b_{ij}^{(k+1)} = \begin{cases} b_{ij}^{(k)}, & i \leqslant k \\ 0, & i \geqslant k+1, j \leqslant k \\ (b_{ij}^{(k)} - s_{ik} b_{kj}^{(k)}(1 + \theta_{ij}^{(k)} 10^{-t}))(1 + \phi_{ij}^{(k)} 10^{-t}), & i \geqslant k+1, j \geqslant k+1 \end{cases}$$

$$s_{ij} = \begin{cases} 0, & i \leqslant j \\ 1, & i = j \\ \frac{b_{ij}^{(j)}}{b_{jj}^{(j)}}(1 + \psi_{ij} 10^{-t}), & i > j \end{cases}$$

here
$$|\theta_{ij}^{(k)}| \leqslant 5, \quad |\phi_{ij}^{(k)}| \leqslant 5, \quad |\psi_{ij}| \leqslant 5$$
and they account for the rounding procedures in floating-point arithmetic. The above calculations can be carried out iff the $b_{jj}^{(j)}$ are nonzero for $j \leqslant n-1$. This can be assured by the following lemma.

Lemma 2.1.3: Numerical Gaussian Elimination

If A is non-singular and t is sufficiently large, then the Gaussian elimination method, with maximal column pivots and floating-point arithmetic (with t-digit mantissas), yields multipliers s_{ij} with $|s_{ij}| \leqslant 1$ and pivots $b_{jj} \neq 0$.

It turns out that we require a bound on the growth of the pivot elements for our error estimates. That is, we seek a quantity $G(n)$, independent of the a_{ij}, such that
$$|b_{jj}^{(j)}| \leqslant G(n)a, \qquad a = \max_{i,j} |a_{ij}| \tag{2.8}$$
It is not difficult to see that
$$G(n) \leqslant (1 + (1 + 5 \times 10^{-t})^2)^{n-1} = 2^{n-1} + \mathcal{O}((n-1)10^{1-t}) \tag{2.9}$$
This establishes the existence of a bound, but it is a tremendous overestimate for large n. In fact for exact elimination, using maximal pivots it can be shown that
$$|a_{jj}^{(j)}| < g(j)a, \qquad g(j) \leqslant 3j^{1/2 + 1/4 \ln j} \tag{2.10}$$
The quantity $g(n)$ would be a reasonable estimate for $G(n)$ if the maximal pivots in the sequence $\{B^{(k)}\}$ were located in the same positions as the maximal pivots in $\{A^{(k)}\}$. We know that if A is non-singular and t is sufficiently large, then the indices of the maximal pivotal elements used to find $\{B^{(k)}\}$ are also indices of maximal pivots in an exact Gaussian elimination procedure for A. (because of continuity)

The best lower bound for $G(n)$ is not known yet.

We now turn to estimates of the terms in δA. Our first result is a bound on the elements of E:

Proposition: **Bounds for E**

Under the hypotheses of lemma 2.1.3, the elements of E satisfy

$$|e_{ij}| \leqslant \begin{cases} (i-1)2aG(n)10^{1-t}, & i \leqslant j \\ j2aG(n)10^{1-t}, & i > j \end{cases} \tag{2.11}$$

For any G satisfying (2.8).

Proof. We can write

$$b_{ij}^{(k+1)} = b_{ij}^{(k)} - s_{ik}b_{kj}^{(k)} + \epsilon_{ij}^{(k+1)}, \qquad i \geqslant k+1, j \geqslant k+1 \tag{2.12}$$

where

$$\epsilon_{ij}^{(k+1)} = b_{ij}^{(k)}\phi_{ij}^{(k)}10^{-t} - s_{ik}b_{kj}^{(k)}(\theta_{ij}^{(k)} + \phi_{ij}^{(k)})10^{-t} - \cdots$$

For maximal pivots, we have $|s_{ik}| \leqslant 1$ and $|b_{ij}^{(k)}| \leqslant G(n)a$, so we have

$$|\epsilon_{ij}^{(k+1)}| \leqslant 2aG(n)10^{1-t}$$

For the second equation

$$s_{ij} = \frac{b_{ij}^{(j)}}{b_{jj}^{(j)}}(1 + \psi_{ij}10^{-t}), \qquad i > j$$

rewrite it as

$$0 = b_{ij}^{(k)} - s_{ij}b_{jj}^{(j)} + \epsilon_{ij}^{(j+1)}, \qquad i > j$$

where

$$\epsilon_{ij}^{(j+1)} = s_{ij}b_{jj}^{(j)}\psi_{ij}10^{-t}$$

where again we find that

$$|\epsilon_{ij}^{(j+1)}| \leqslant 2aG(n)10^{1-t}$$

Now, we can write $\mathcal{L} = (s_{ij})$ and $\mathcal{U} = (b_{ij}^{(i)})$. We have

$$(\mathcal{LU})_{ij} = \sum_{k=1}^{n} s_{ik}b_{kj}^{(k)} = \sum_{k=1}^{\min(i,j)} s_{ik}b_{kj}^{(k)}$$

For $i > j$, as we have

$$0 = b_{ij}^{(1)} - \sum_{k=1}^{j} s_{ik}b_{kj}^{(k)} + \sum_{k=1}^{j} \epsilon_{ij}^{(k+1)}$$

from the first j steps of the Gaussian elimination procedure (2.12), we have

$$e_{ij} = \sum_{k=1}^{j} \epsilon_{ij}^{(k+1)}, \qquad i > j$$

For $i \leqslant j$, we have

$$e_{ij} = \sum_{k=1}^{i-1} \epsilon_{ij}^{(k+1)}, \qquad i \leqslant j$$

giving the desired bounds. □

As a simple corollary of this theorem, we note that since $|e_{ij}| \leqslant 2naG(n)10^{1-t}$, we have

$$\|E\|_\infty \leqslant 2an(n-1)G(n)10^{1-t}$$

The elements in $\delta\mathcal{L}$ and $\delta\mathcal{U}$ can be estimated from a single analysis of the error in solving any triangular system (with the same arithmetic and rounding). Thus we consider, say, $Tu = h$, where $T = (t_{ij})$ is lower triangular and nonsingular. Then we have

$$u_1 = t_{11}^{-1}h_1, \quad u_i = t_{ii}^{-1}\left(h_i - \sum_{j=1}^{i-1} t_{ij}u_j\right), \quad i = 2, 3, \ldots, n \tag{2.13}$$

Proposition: **Bounds of Back Substitution Errors**

Let the solution of (2.13) be computed with floating-point arithmetic with t-digit mantissas, then the computed solution v satisfies $(T + \delta T)v = h$, where δT is bounded by

$$|\delta t_{ij}| \leqslant \max\{2, |i - j + 1|\}|t_{ij}|10^{1-t} \leqslant n|t_{ij}|10^{1-t}$$

Here t is required to be sufficiently large so that $n10^{1-t} \leqslant 1$.

We are now able to obtain estimates of the elements in $\delta\mathcal{L}$ and $\delta\mathcal{U}$, and thus of δA.

Theorem 2.1.4: Wilkinson Error Estimate

Let the n-th order matrix A be non-singular and employ Gaussian elimination with maximal pivots and t-digit floating point arithmetic to solve $Ax = f$. Let t be sufficiently large so that $n10^{1-t} \leqslant 1$. Then the computed solution $x + \delta x$ satisfies

$$(A + \delta A)(x + \delta x) = f$$

where

$$|\delta a_{ij}| \leqslant (2n + 3n^2)aG(n)10^{1-t} \tag{2.14}$$

and $G(n)$ is any bound satisfying (2.8).

Proof. SORRY. □

From this theorem, it follows that the computed solution is the exact solution of a system only slightly perturbed from the original if enough figures are used. Appropriate values for t depend upon n and the bound $G(n)$. If $G(n) \sim 2^{n-1}$, only relatively small order systems could be treated effectively. On the other hand, if $G(n) \sim g(n) \leqslant 3n^{1/2+1/4\ln n}$, then quite large order systems can be treated with the number of digits used on modern digital computers, say $t \geqslant 8$. It is generally believed, however, that even this latter estimate for $G(n)$ is a gross overestimate when using maximal pivots.

It should be observed that essentially all of the previous analysis is valid if only partial pivoting, say maximal column pivoting, is employed since then $|s_{ij}| \leqslant 1$ is maintained. However, the growth factor $G(n)$ for this procedure cannot be estimated well in general. In fact, it is possible that the upper bound (2.9) which still applies may be attained. In spite of this, partial pivoting is found to be effective in practice but the absence of any type of maximal pivoting strategy frequently leads to catastrophic growth of rounding errors.